11 source

Most programming languages permit programmers to specify external files to be included within their programs. This is often used to add “boilerplate” code to programs for such things as defining standard constants and referencing external library function definitions.

Bash (along with `ksh` and `zsh`) has a builtin command, `source`, that implements this feature. We looked at `source` briefly when we worked with the `.profile` and `.bashrc` files used to establish the shell environment.

In this adventure, we will look at `source` again and discover the ways it can make our scripts more powerful and easier to maintain.

To recap, `source` reads a specified file and executes the commands within it using the current shell. It works both with the interactive command line and within a script. Using the command line for example, we can reload the `.bashrc` file by executing the following command:

```
me@linuxbox: ~$ source ~/.bashrc
```

Note that the `source` command can be abbreviated by a single dot character like so:

```
me@linuxbox: ~$ . ~/.bashrc
```

When `source` is used on the command line, the commands in the file are treated as if they are being typed directly at the keyboard. In a shell script, the commands are treated as though they are part of the script.

Configuration Files

During our exploration of the Linux ecosystem, we have seen that many programs rely on configuration files. Most of these are simple text files just like our bash shell scripts. By using `source`, we can easily create configuration files for our shell scripts as well.

Consider this example. Let’s imagine that we have several computers on our network that need to get backed up on a regular basis and that a central backup server is used to store the files from these various systems. On each of the backup client systems we have a script called `back_me_up` that copies the files over the network. Let’s further imagine that each client system needs to back up a different set directories.

To implement this, we might define a constant in the `back_me_up` script like this:

```
BACKUP_DIRS="/etc /usr/local /home"
```

However, doing it this way will require that we maintain a separate version of the script for each client. This will make maintaining the script much more laborious, as any future improvement to the script will have to be applied to each copy of the script individually.

What's more, this list of directories might be useful to other programs, too. For example, we could have a file restoration script called `restore_me` that restores files from the backup server to the backup client system. If this were the case, we would then have twice as many scripts to maintain. A much better way handle this issue would be to create a configuration file to define the `BACKUP_DIR` constant and source it into our scripts at run time.

Here's how we could do it.

First, we will create a configuration file named `back_me_up.cfg` and place it somewhere sensible. Since the `back_me_up` and `restore_me` scripts are used on a system-wide basis (as would most backup programs), we will treat them like locally installed system resources. Thus, we would put them in the `/usr/local/sbin` directory and the configuration file in `/usr/local/etc`. The configuration file would contain the following:

```
# Configuration file for the back_me_up program

BACKUP_DIRS="/etc /usr/local /home"
```

While our configuration file must contain valid shell syntax, since its contents are executed by the shell, it differs from a real shell script in two regards. First, it does not require a shebang to indicate which shell executes it, and second, the file does not need executable permissions. It only needs to be readable by the shell.

Next, we would add the following code to the `back_me_up` and `restore_me` scripts to source our configuration file:

```
CONFIG_FILE=/usr/local/etc/back_me_up.cfg

if [[ -r "$CONFIG_FILE" ]]; then
    source "$CONFIG_FILE"
else
    echo "Cannot read configuration file!" >&2
    exit 1
fi
```

Function Libraries

In addition to the configuration shared by both the `back_me_up` and `restore_me` scripts, there could be code shared between the two programs. For example, it makes sense to have a shared function to display error messages:

```
# ----------------------------------------------------------------------------
# Send message to std error
#   Options:    none
#   Arguments:  1 error_message
#   Notes:      Use this function to report errors to standard error. Does
#               not generate an error status.
# ----------------------------------------------------------------------------
error_msg() {
```

```
printf "%s\n" "$1" >&2
}
```

How about a function that detects if the backup server is available on the network:

```
# ---------------------------------------------------------------------------
# Detect if the specified host is available
#   Options:    none
#   Arguments:  1 hostname
#   Notes:
# ---------------------------------------------------------------------------
ping_host() {

  local MSG="Usage: ${FUNCNAME[0]} host"
  local MSG2="${FUNCNAME[0]}: host $1 unreachable"

  [[ $# -eq 1 ]] || { error_msg "$MSG" ;return 1; }
  ping -c1 "$1" &> /dev/null || { error_msg "$MSG2"; return 1; }
  return 0
}
```

Another function both scripts could use checks that external programs we need for the scripts to run (like `rsync`) are actually installed:

```
# ---------------------------------------------------------------------------
# Check if function/application is installed
#   Options:    none
#   Arguments:  application...
#   Notes:      Exit status equals the number of missing functions/apps.
# ---------------------------------------------------------------------------
app_avail() {

  local MSG1="Usage: ${FUNCNAME[0]} app..."
  local MSG2
  local exit_status=0

  [[ $# -gt 0 ]] || { error_msg "$MSG1"; return 1; }
  while [[ -n "$1" ]]; do
    MSG2="Required program '$1' not available - please install"
    type -t "$1" > /dev/null || \
      { error_msg "$MSG2"; exit_status=$((exit_status + 1)); }
    shift
  done
  return "$exit_status"
}
```

To share these functions between the `back_me_up` and `restore_me` scripts, we could build a library of functions and source that library. As an example, we could put all the common code in a file called `/usr/local/lib/bmulib.sh` and add the following code to both scripts to source that file:

```
FUNCLIB=/usr/local/lib/bmulib.sh

if [[ -r "$FUNCLIB" ]]; then
    source "$FUNCLIB"
else
    echo "Cannot read function library!" >&2
    exit 1
fi
```

General Purpose Libraries

Since we hope to become prolific script writers, it makes sense over time, to build a library of common code that our future scripts could potentially use. When undertaking such a project, it's wise to write high quality functions for the library, as the code may get heavy use. It's important to test carefully, include a lot of error handling, and fully document the functions. After all, the goal here is to save time writing good scripts, so invest the time up front to save time later.

Let's Not Forget .bashrc

`source` can be a powerful tool for coordinating the configuration of small sets of machines. For large sets, there are more powerful tools, but `source` works fine if the job is not too big.

We've worked with the `.bashrc` file before and added things like aliases and a few shell functions. However, when we work with multiple systems (for example, a small network), it might be a good idea to create a common configuration file to align all of the systems. To demonstrate, let's create a file called `.mynetworkrc.sh` and place all of the common aliases and shell function we would expect on every machine. To use this file, we would add this one line of code to `.bashrc`:

```
[[ -r ~/.mynetworkrc.sh ]] && source ~./.mynetworkrc.sh
```

The advantage of doing it this way is that we won't have to cut and paste large sections of code every time we configure a new machine or perform an operating system upgrade. We just copy the `.mynetwrokrc.sh` file to the new machine and add one line to `.bashrc`.

We can even go further and create a host-specific configuration file that the `.mynetworkrc.sh` file will source. This would be handy if we need to override something in `.mynetworkrc.sh` on a particular host. We can do this by creating a configuration file with a file name based on the system's host name. For example, if our system's host name is `linuxbox1` we could create a configuration file named `.linuxbox1rc.sh` and add this line of code to the `.mynetworkrc.sh` file:

```
[[ -r ~/.$(hostname)rc.sh ]] && source ~/.$(hostname)rc.sh
```

By using the `hostname` command we are able to build a file name that is specific to a particular host.

So, what could we put in our `.mynetworkrc.sh` file? Here are some ideas:

```
### Aliases ###

# Reload the .mynetworkrc.sh file. Handy after editing.
alias reload='source ~/.mynetworkrc.sh'

# Get a root shell
```

```
alias root='sudo -i'

# Print the size of a terminal window in rows and columns
alias term_size='echo "Rows=$(tput lines) Cols=$(tput cols)"'

### Functions ###

# Check to see if a specified host is alive on the network

ping_host() {

  local target

  if [[ -z "$1" ]]; then
    echo "Usage: ping_host host" >&2
    return 1
  fi
  target="$1"
  ping -c1 "$target" &> /dev/null || \
    { echo "Host '$target' unreachable." >&2; return 1; }
  return 0
}

# Display a summary of system health

status() {
  { # Display system uptime
    echo -e "\nuptime:"
    uptime

    # Display disk resources skipping snap's pseudo disks
    echo -e "\ndisk space:"
    df -h 2> /dev/null | grep -v snap
    echo -e "\ninodes:"
    df -i 2> /dev/null | grep -v snap
    echo -e "\nblock devices:"
    /bin/lsblk | grep -v snap

    # Display memory resources
    echo -e "\nmemory:"
    free -m

    # Display latest log file entries
    if [[ -r /var/log/syslog ]]; then # Ubuntu
      echo -e "\nsyslog:"
      tail /var/log/syslog
    fi
    if [[ -r /var/log/messages ]]; then # Debian, et al.
      echo -e "\nmessages:"
      tail /var/log/messages
    fi
    if [[ -r /var/log/journal ]]; then # Arch, others using systemd
      echo -e "\njournal:"
      journalctl | tail
    fi
  } | less
}

# Install a package from a distro repository
# Supports Ubuntu, Debian, Fedora, CentOS
```

```
install() {
  if [[ -z "$1" ]]; then
    echo "Usage: install package..." >&2
    return 1
  elif [[ "$1" == "-h" || "$1" == "--help" ]]; then
    echo "Usage: install package..."
    return
  elif [[ -x /usr/bin/apt ]]; then
    sudo apt update && sudo apt install "$@"
    return
  elif [[ -x /usr/bin/apt-get ]]; then
    sudo apt-get update && sudo apt-get install "$@"
    return
  elif [[ -x /usr/bin/yum ]]; then
    sudo yum install -y "$@"
  fi
}

# Perform a system update
# Supports Debian, Ubuntu, Fedora, CentOS, Arch

update() {
  if [[ -x /usr/bin/apt ]]; then # Debian, et al
    sudo apt update && sudo apt upgrade
    return
  elif [[ -x /usr/bin/apt-get ]]; then # Old Debian, et al
    sudo apt-get update && sudo apt-get upgrade
    return
  elif [[ -x /usr/bin/yum ]]; then # CentOS/Fedora
    # su -c "yum update"
    sudo yum update
    return
  elif [[ -x /usr/bin/pacman ]]; then # Arch
    sudo pacman -Syu
  fi
}

# Display distro release info (prints OS name and version)

version() {

  local s

  for s in os-release \
           lsb-release \
           debian_version \
           centos-release \
           fedora-release; do
    [[ -r "/etc/$s" ]] && cat "/etc/$s"
  done
}
```

Ever Wonder Why it's Called .bashrc?

In our various wanderings around the Linux file system, we have encountered files with names that end with the mysterious suffix "rc" like `.bashrc`, `.vimrc`, etc. Heck, many distributions have a bunch of directories in `/etc` named rc. Why is that? It's a holdover from ancient Unix. Its original meaning was "run commands," but it later became "run-

control." A run-control file is generally some kind of script or configuration file that prepares an environment for a program to use. In the case of `.bashrc` for example, it's a script that prepares a user's bash shell environment.

Security Considerations and Other Subtleties

Even though sourced files are not directly executable, they do contain code that will be executed by anything that sources them. It is important, therefore, that permissions be set to allow writing only by their owners.

```
me@linuxbox:~$ sudo chmod 644 /usr/local/etc/back_me_up.cfg
```

If a sourced file contains confidential information (as a backup program might), set the permissions to 600.

While `bash`, `ksh`, and `zsh` all have the `source` builtin, `dash` and all other strictly POSIX compatible shells support only the single dot (.).

If the file name argument given to `source` does not contain a / character, the directories listed in the `PATH` variable are searched for the specified file. For security reasons, it's probably not a good idea to rely on this. Always specify a explicit path name.

Another subtlety has to do with positional parameters. As `source` executes its commands in the current shell environment, this includes the positional parameters a script was given as well. This is fine in most cases; however, if `source` is used within a shell function and that shell function has its own positional parameters, `source` will ignore them and use the shell's environment instead. To overcome this, positional parameters may be specified after the file name. Consider the following script:

```
#!/bin/bash

# foo_script

source ~/foo.sh

foo() {
  source ~/foo.sh
}

foo "string2"
```

`foo.sh` contains this one line of code:

```
echo $1
```

We expect to see the following output:

```
me@linuxbox: ~$ foo_script string1
string1
string2
```

But, what we actually get is this:

```
me@linuxbox: ~$ foo_script string1
string1
string1
```

This is because `source` uses the shell environment the script was given, not the one that exists when the function called. To correct this, we need to write our script this way:

```
#!/bin/bash

# foo_script

source ~/foo.sh

foo() {
  source ~/foo.sh "$1"
}

foo "string2"
```

By adding the desired parameter to the `source` command within the function `foo`, we are able to get the desired behavior. Yes, it's subtle.

Summing Up

By using `source`, we can greatly reduce the effort needed to maintain our bash scripts particularly when we are deploying them across multiple machines. It also allows us to effectively reuse code with function libraries that all of our scripts can share. Finally, we can use `source` to build much more capable shell environments for our day to day command line use.

Further Reading

- A Wikipedia article on the dot command from which the `source` builtin is derived: https://en.wikipedia.org/wiki/Dot_(command)
- Another article about run-commands: https://en.wikipedia.org/wiki/Run_commands